# AP Computer Science Principles

## Week 9: Writing Functions

### Today's Objectives:

- Students will be able to define a function and call it.
- Students will be able to explain the difference between a parameter and an argument.
- Students will be able to trace a program that calls functions.
- Students will be able to test a function and use error messages to debug it.

# Why Functions?

- A function is a named block of code that does one job.
- Write it once, then use it as many times as you need.
- Functions break a big program into smaller pieces that are easier to read and fix.
- You've already been using functions: `Rect()`, `Circle()` and `print()` are functions someone else wrote.

# Defining a Function

- The keyword `def` starts a function definition.
- Then the name, parentheses `( )`, and a colon.
- The body is indented, with every line lined up the same way. Most people use 4 spaces.
- Defining a function does not run it. Nothing is drawn yet!

```
def drawTree():
    Rect(190, 250, 20, 80, fill='sienna')
    Circle(200, 230, 40, fill='green')
```

# Calling a Function

- To run a function, call it: write its name followed by `()`.

Code

```
def greet():
    print('Welcome to CSP!')
    print('Let us code.')

greet()
greet()
```

Console

```
Welcome to CSP!
Let us code.
Welcome to CSP!
Let us code.
```

# Parameters

- A parameter is a variable in the definition that receives a value.
- An argument is the actual value you pass in when you call the function.
- `drawTree(100)` sets `x` to 100 for that call. `drawTree(300)` sets it to 300.

```
def drawTree(x):
    Rect(x - 10, 250, 20, 80, fill='sienna')
    Circle(x, 230, 40, fill='green')

drawTree(100)
drawTree(300)
```

# Tracing Function Calls

- Python skips over the definition until the function is called.
- When the function finishes, Python returns to the line after the call.

Code

```
def cheer(team):
    print('Go', team + '!')

print('Game day')
cheer('Hawks')
cheer('Owls')
print('Final score')
```

Console

```
Game day
Go Hawks!
Go Owls!
Final score
```

# Testing Functions

- CMU's autograder tests your function by calling it with different arguments and checking the result.
- Test it yourself before you submit:
  - typical values (like 100 or 250)
  - edge cases: values at the limits, like 0 or 400
  - values someone might not expect
- If a test fails, compare what you expected with what actually happened.

# Debugging Functions

| Mistake | What you'll see |
| --- | --- |
| `def greet()` (no colon) | SyntaxError |
| Body not indented | IndentationError |
| Calling `greet()` before the `def` | NameError |
| `Greet()` when the name is `greet` | NameError (names are case-sensitive) |
| `greet` without `()` | No error, but nothing happens |

# Check Your Understanding

Think, then share

1. What is the difference between defining a function and calling it?
2. In `drawTree(150)`, what is the parameter and what is the argument?
3. The line `def drawSun():` is followed by a line that isn't indented. What happens?
4. A program defines `cheer` and then calls `cheer('A')`, `cheer('B')`, `cheer('C')`. How many times does the body run?

# Try It: Welcome Banner

CMU Sandbox

- Define a function `printBanner()` that prints a 3-line banner.
- Use at least two escape sequences (like `\t` and `\"`).
- Call your function twice, with a `print()` between the calls.

**Sample output**

```
************************
*      "Welcome to CSP"
************************

************************
*      "Welcome to CSP"
************************
```

*Adapted from River Hill CompSci, Python – Output exercises*

# Try It: Shape Functions

CMU Sandbox

- Define `printDiamond()` that prints a filled diamond out of `*` characters (at least 5 lines). Call it.
- Then define `drawBadge(x)` that draws a `Star` centered at (x, 200).
- Call `drawBadge` three times so three badges appear across the canvas.

Sample output

```
  *
 ***
*****
 ***
  *
```

*Adapted from River Hill CompSci, Python – Output exercises*